



Projet HPC - Problème de Poisson 2D/3D

Auteurs :

Hugo CHAZALON
Romain LAPOUGE

Enseeiht - Mécanique des Fluides, Énergétique & Environnement (MF2E)

Superviseur : Reynier Loïc

Décembre 2025

Table des matières

1	Introduction	1
2	Présentation du Problème et Méthode de Résolution	1
2.1	Le problème continu	1
2.2	Discretisation et Méthode Numérique	1
2.3	Stratégie de Parallélisation (SPMD)	2
2.4	Communications et Points Fantômes	2
3	Calcul Parallèle	2
3.1	Configuration Matérielle (Machine Turpan)	2
3.2	Tests de convergence sur petits maillages	3
3.3	Tests de performance	3
4	Cas 3D : Analyse de Scalabilité (CPU)	4
4.1	Graphiques de Scalabilité et d'Énergie	4
4.2	Analyse	4
5	Cas 3D : Analyse de Scalabilité (GPU)	5
5.1	Graphiques de Performance (Énergie et Speed-up)	5
5.2	Scalabilité et Temps de Restitution	5
5.3	Consommation Énergétique et Variabilité	5
5.4	Bilan	6
6	Comparaison des Architectures : CPU vs GPU	6
6.1	Bilan comparatif	6

Table des figures

1	Domaine discrétisé et CL.	1
2	Stencil à 5 points (Jacobi).	1
3	Décomposition en 4 domaines.	2
4	Communications (Ghost Cells).	2
5	Temps de calcul 2D.	4
6	Speed-up 2D.	4
7	Speed-up 3D.	4
8	Consommation 3D.	4
9	Speed-up.	5
10	Énergie.	5
11	Comparaison des Temps.	6
12	Comparaison Énergie (Plus bas est mieux).	6

1 Introduction

Ce projet prend par dans le cadre du cours 'Environnement Logiciel du Calcul Scientifique'. Ce projet à pour objectif de se familiariser avec le calcul haute performance (HPC) en utilisant le super calculateur de cal mip.

Pour se faire nous allons résoudre un problème de Poisson 2D et 3D en utilisant plusieurs configuration de calcul sur des maillage plus au moins précis avec une puissance de calcul plus ou moins grosse.

Durant ce projet, nous allons mettre en valeur l'importance de bien configurer le super calculateur de façon à optimiser le temps de calcul et à ne pas perdre de temps à échanger des données inutilement.

2 Présentation du Problème et Méthode de Résolution

Ce projet s'intéresse à la résolution numérique du problème de Poisson en deux dimensions (2D) sur un domaine carré, ainsi qu'à sa parallélisation via le standard MPI.

2.1 Le problème continu

Nous cherchons à résoudre l'équation de Poisson stationnaire sur un domaine $\Omega = [0, 1] \times [0, 1]$. Il ne s'agit pas d'un problème d'évolution temporelle. Le problème se formule ainsi :

$$\begin{cases} \Delta u(x, y) = f(x, y) & \text{dans } \Omega \\ u(x, y) = 0 & \text{sur } \partial\Omega \end{cases} \quad (1)$$

où $f(x, y) = 2(x^2 - x + y^2 - y)$ est le terme source connu et $u(x, y)$ est l'inconnue. La solution exacte est connue et donnée par $u_{exact} = xy(x-1)(y-1)$, ce qui nous permettra de valider la précision de notre méthode numérique.

2.2 Discrétisation et Méthode Numérique

Le domaine continu est discrétisé spatialement à l'aide d'une grille (maillage) comportant ntx points intérieurs en x et nty points intérieurs en y (avec $ntx = nty$). Le pas de discrétisation est défini par $h = \frac{1}{ntx+1}$.

Pour résoudre ce système, nous utilisons la méthode des différences finies couplée à une méthode itérative de Jacobi (ordre $O(h^2)$). À chaque itération $n+1$, la valeur en un point (i, j) est calculée à partir des valeurs de ses quatre voisins (Nord, Sud, Est, Ouest) à l'itération précédente n :

$$u_{ij}^{n+1} = c_0 [c_1(u_{i+1,j}^n + u_{i-1,j}^n) + c_2(u_{i,j+1}^n + u_{i,j-1}^n) - f_{ij}] \quad (2)$$

avec les coefficients $c_0 = \frac{h^2}{4}$ et $c_1 = c_2 = \frac{1}{h^2}$. Cette dépendance locale est illustrée par le stencil à 5 points.

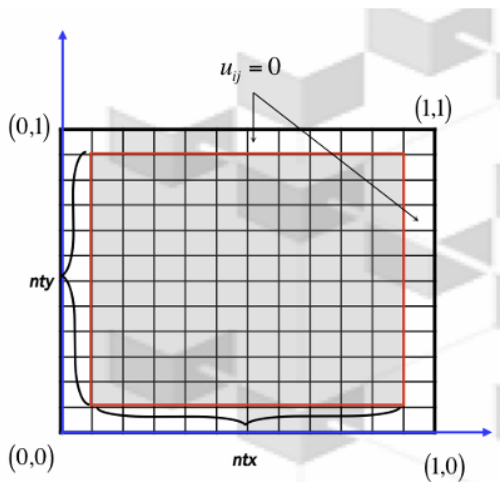


FIGURE 1 – Domaine discrétisé et CL.

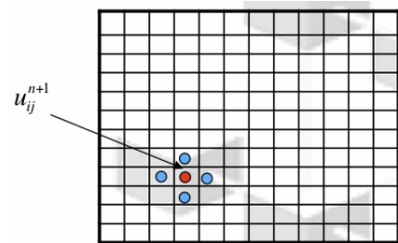


FIGURE 2 – Stencil à 5 points (Jacobi).

2.3 Stratégie de Parallélisation (SPMD)

Pour accélérer les calculs, nous adoptons une approche SPMD (*Single Program Multiple Data*). Le domaine global est découpé géométriquement en sous-domaines, chacun étant attribué à un processus MPI distinct.

Dans le cas 2D traité ici, la décomposition s'effectue par "tranches" horizontales (découpage selon l'axe Y). Si nous utilisons NP processus, chaque processus gère un sous-domaine de taille $(\frac{nty}{NP}) \times ntx$. Cela permet d'équilibrer la charge de calcul entre les processus.

2.4 Communications et Points Fantômes

La méthode de Jacobi nécessitant la valeur des voisins pour mettre à jour un point, les points situés aux frontières d'un sous-domaine ont besoin de données appartenant aux processus voisins (Nord et Sud).

Pour gérer cela, chaque sous-domaine local est étendu avec des lignes supplémentaires appelées "points fantômes" (ou *ghost cells*).

- Une ligne fantôme au Sud reçoit les données de la dernière ligne du voisin $p - 1$.
- Une ligne fantôme au Nord reçoit les données de la première ligne du voisin $p + 1$.

À chaque itération, une phase de communication est nécessaire : chaque processus envoie ses données de bord à ses voisins et réceptionne les leurs dans ses zones fantômes avant d'effectuer le calcul.

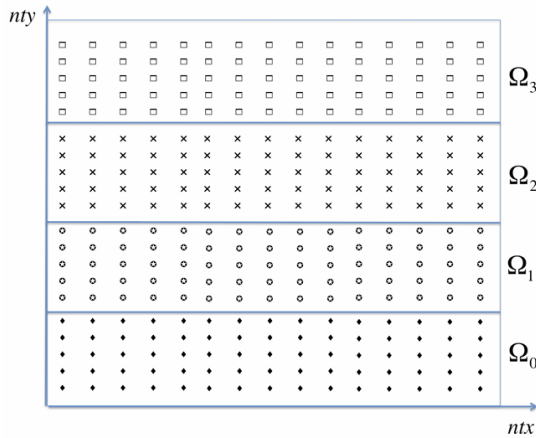


FIGURE 3 – Décomposition en 4 domaines.

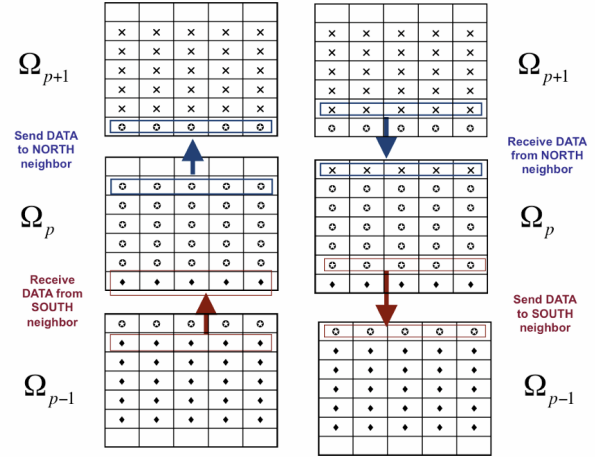


FIGURE 4 – Communications (Ghost Cells).

3 Calcul Parallèle

3.1 Configuration Matérielle (Machine Turpan)

Les tests de performance sont effectués sur le supercalculateur **Turpan**. Chaque nœud de calcul est équipé d'un processeur **Ampere Altra Q80-30** basé sur l'architecture ARM v8.2.

Les caractéristiques du processeur (socket) sont les suivantes :

- **Nombre de cœurs** : 80 cœurs physiques.
- **Fréquence (f)** : 3.0 GHz.
- **Système d'exploitation** : Environnement Linux.

Performance théorique (R_{peak}) : Sachant qu'un cœur ARM Neoverse N1 (architecture de l'Ampere Altra) peut exécuter 8 opérations flottantes en double précision par cycle (2 unités SIMD $\times$ 2 DP $\times$ FMA), la performance crête théorique d'un socket est donnée par :

$$R_{peak} = \text{Nb_Cœurs} \times \text{Fréquence} \times \text{FLOPs/cycle} \quad (3)$$

$$R_{peak} = 80 \times 3.0 \times 10^9 \times 8 = 1920 \text{ GFlop/s} = 1.92 \text{ TFlop/s} \quad (4)$$

3.2 Tests de convergence sur petits maillages

Nous avons réalisé des tests pour des tailles de maillage $NTX \in \{16, 32, 64\}$ en faisant varier le nombre de processus $NP \in \{1, 2, 4\}$. Les résultats obtenus (nombre d'itérations pour converger et erreur globale finale) sont consignés dans le tableau ci-dessous.

Taille (NTX)	Processus (NP)	Itérations	Erreur Globale	Temps (s)
16	1	947	9.89×10^{-11}	4.88×10^{-3}
	2	947	9.89×10^{-11}	3.28×10^{-3}
	4	947	9.89×10^{-11}	3.19×10^{-3}
32	1	3288	9.99×10^{-11}	6.25×10^{-2}
	2	3288	9.99×10^{-11}	3.44×10^{-2}
	4	3288	9.99×10^{-11}	2.38×10^{-2}
64	1	11609	1.00×10^{-10}	0.879
	2	11609	1.00×10^{-10}	0.465
	4	11609	1.00×10^{-10}	0.266

TABLE 1 – Convergence de la méthode de Jacobi pour différentes tailles de problèmes et nombres de processus.

Analyse de la convergence : On observe que pour une taille de problème donnée (NTX fixé), le nombre d'itérations nécessaires pour atteindre la convergence est strictement identique, quel que soit le nombre de processus utilisés. De même, l'erreur globale finale est identique. Cela valide notre implémentation parallèle : la décomposition de domaine et la gestion des points fantômes n'altèrent pas le résultat numérique par rapport à une exécution séquentielle.

Relation entre taille du problème et itérations : Lorsque l'on double la taille du maillage (par exemple de $NTX = 16$ à 32 , ou de 32 à 64), le nombre d'itérations augmente considérablement.

— Passage de 16 à 32 : le nombre d'itérations est multiplié par environ 3.5 (3288/947).

— Passage de 32 à 64 : le nombre d'itérations est multiplié par environ 3.5 (11609/3288).

Théoriquement, pour la méthode de Jacobi appliquée au problème de Poisson discrétisé, le nombre d'itérations nécessaire pour réduire l'erreur d'un facteur donné est proportionnel à $O(N^2)$. Nos résultats expérimentaux sont cohérents avec cette propriété (un facteur 4 théorique pour un doublement de la taille). Cette augmentation quadratique du coût souligne les limites de la méthode de Jacobi pour de très grands systèmes.

3.3 Tests de performance

Pour évaluer le gain de performance apporté par la parallélisation, nous utilisons une métrique appelée le **Speed-up** (ou accélération). Il se définit comme le rapport entre le temps d'exécution séquentiel (avec un seul processus) et le temps d'exécution parallèle (avec N processus) :

$$S(N) = \frac{T_{ref}}{T_N} \quad (5)$$

où T_{ref} est le temps sur 1 processus et T_N le temps sur N processus.

— Si $S(N) \approx N$, l'accélération est dite *idéale* ou *linéaire* : utiliser N fois plus de ressources divise le temps par N .

— Si $S(N) < N$, le gain est sous-linéaire, ce qui indique une perte d'efficacité (due aux communications ou à l'accès mémoire).

Les deux graphiques ci-dessous représentent l'évolution du temps de calcul et du Speed-up en fonction du nombre de processus pour nos trois tailles de maillage ($NTX = 5120, 10240, 20480$).

L'analyse des courbes de Speed-up montre deux comportements distincts :

1. **Jusqu'à 40 processus :** L'accélération est excellente et quasi-identique à la courbe idéale (linéaire) pour toutes les tailles de maillage. Cela signifie que le découpage du travail est efficace et que le coût des communications reste négligeable.

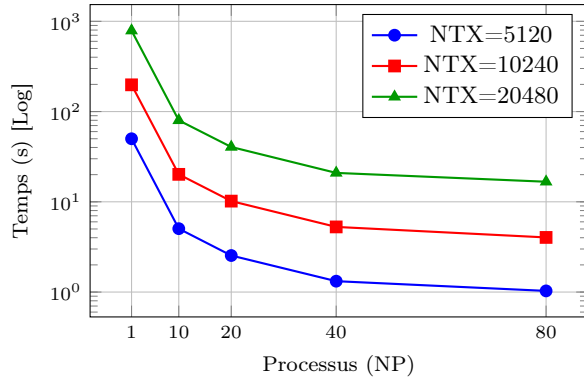


FIGURE 5 – Temps de calcul 2D.

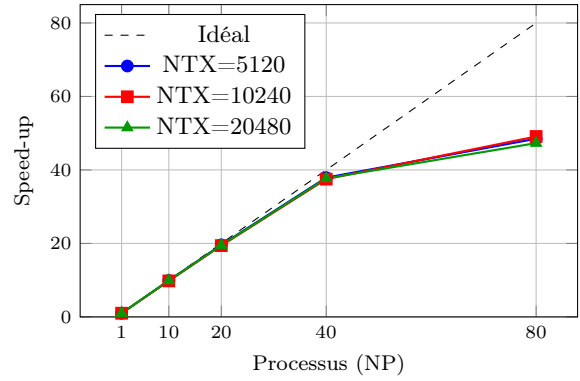


FIGURE 6 – Speed-up 2D.

2. À 80 processus : On observe un décrochage net. Le Speed-up plafonne autour de 48 au lieu des 80 attendus. Puisque nous utilisons tous les cœurs d'un même nœud, cette baisse de performance s'explique probablement par la saturation de la bande passante mémoire. Les 80 cœurs tentent d'accéder simultanément à la mémoire vive, créant un goulot d'étranglement qui ralentit le calcul global.

4 Cas 3D : Analyse de Scalabilité (CPU)

Nous avons effectué des mesures de scalabilité forte sur le code 3D (MPI pur) en faisant varier la taille du problème ($Nx \in \{504, 1032, 2040\}$) et le nombre de processus. Les calculs sont répartis sur des nœuds complets de 64 cœurs (Partition Q80-30).

Les configurations testées sont :

- 1 Nœud (64 processus) : décomposition $4 \times 4 \times 4$.
- 8 Nœuds (512 processus) : décomposition $8 \times 8 \times 8$.
- 12 Nœuds (768 processus) : décomposition $8 \times 12 \times 8$.

4.1 Graphiques de Scalabilité et d'Énergie

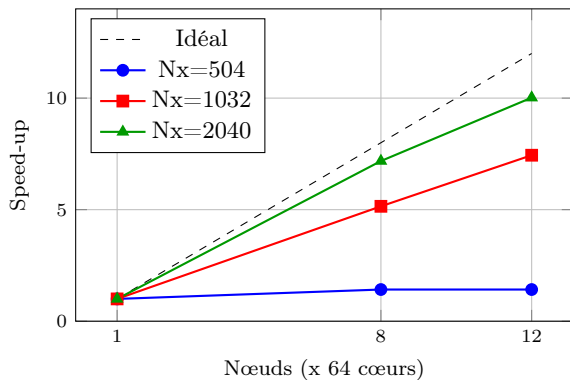


FIGURE 7 – Speed-up 3D.

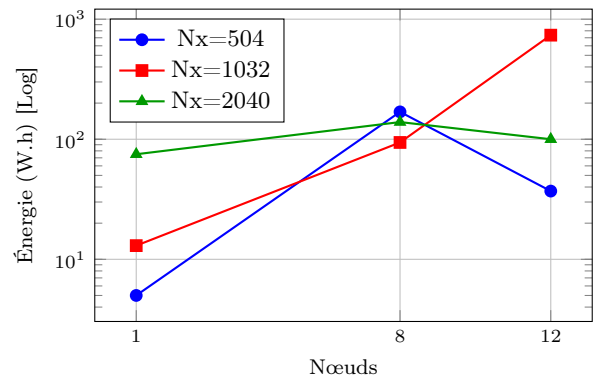


FIGURE 8 – Consommation 3D.

4.2 Analyse

Scalabilité : On observe deux comportements très différents selon la taille du problème :

- Pour le **petit maillage** ($N = 504$), la scalabilité est très mauvaise. Le speed-up sature immédiatement (1.4x pour 8 nœuds). Le temps de calcul est si court (quelques secondes) que le coût des communications MPI domine totalement l'exécution.
- Pour le **grand maillage** ($N = 2040$), la scalabilité est excellente. Nous obtenons un speed-up de ≈ 10 sur 12 nœuds (efficacité de 83%). Ici, la charge de calcul locale est suffisante pour masquer les latences de communication.

Énergie : L'énergie consommée tend à augmenter avec le nombre de nœuds, car l'efficacité parallèle n'est jamais de 100% (on utilise plus de machines pour faire le même travail total). Cependant, pour le cas $N = 2040$, l'augmentation est modérée (de 75 à 139 W.h), et on observe même une baisse à 12 nœuds (100 W.h), ce qui suggère un point de fonctionnement très efficace où la réduction du temps de calcul compense la puissance électrique supplémentaire des nœuds ajoutés. *Note : Certaines valeurs d'énergie pour les petits cas (ex : 736 W.h pour $N=1032$ sur 12 nœuds) semblent aberrantes et pourraient être dues à des artefacts de mesure sur des temps d'exécution très courts.*

5 Cas 3D : Analyse de Scalabilité (GPU)

5.1 Graphiques de Performance (Énergie et Speed-up)

Pour analyser l'efficacité énergétique en parallèle, nous présentons ci-dessous l'évolution de l'énergie consommée et du Speed-up en fonction du nombre de nœuds (1 nœud = 64 cœurs).

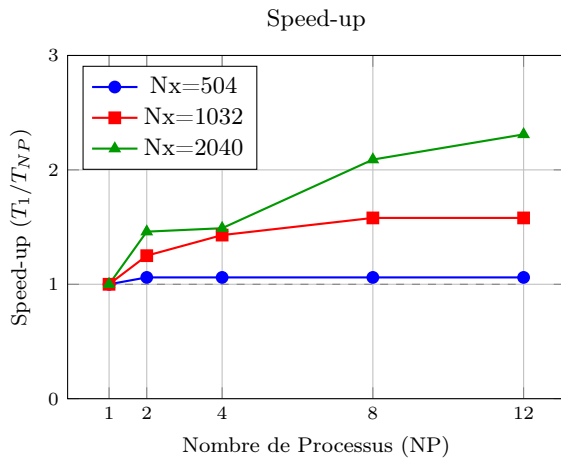


FIGURE 9 – Speed-up.

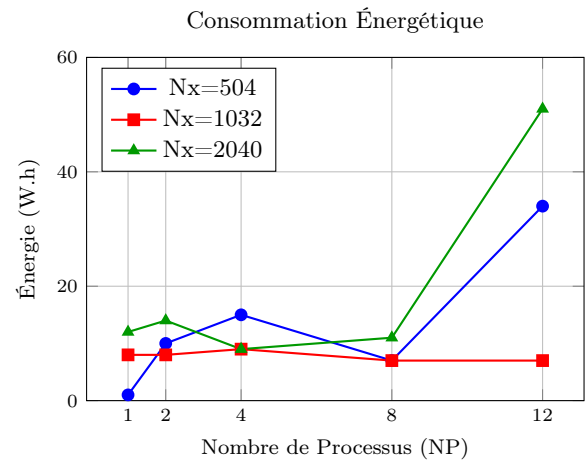


FIGURE 10 – Énergie.

5.2 Scalabilité et Temps de Restitution

Contrairement à une situation de saturation immédiate, nous observons ici un gain de performance réel lorsque la taille du problème est suffisante pour masquer les latences de communication.

- **Pour le petit maillage ($Nx = 504$) :** Le temps de calcul stagne autour de 18-19 secondes quel que soit le nombre de processus. Le régime est dominé par les communications ; paralléliser n'apporte aucun gain ($Speed-up \approx 1$).
- **Pour le grand maillage ($Nx = 2040$) :** L'ajout de ressources permet de réduire le temps de calcul, qui passe de 67s (1 processus) à 29s (12 processus). Le $Speed-up$ atteint ainsi **2.3**, ce qui valide l'intérêt de la parallélisation pour cette taille de problème, même si l'efficacité parallèle reste en deçà de l'idéal linéaire (indiquant des pertes dues aux transferts mémoire ou à la synchronisation).

5.3 Consommation Énergétique et Variabilité

L'analyse de la consommation énergétique révèle une tendance globale à la hausse avec le nombre de processus (passant de ≈ 12 W.h à 51 W.h pour $Nx = 2040$). Cela s'explique par le fait que l'on mobilise davantage de ressources physiques pour un temps de réduction qui n'est pas divisé d'autant.

Note importante sur la variabilité : Il convient de souligner que les mesures énergétiques présentent une certaine instabilité. Pour des configurations similaires (ex : $Nx = 1032$), l'énergie relevée oscille parfois sans corrélation directe avec le temps de calcul (valeurs stables autour de 7-9 W.h malgré l'augmentation de NP). Cette dispersion peut être attribuée à la fréquence d'échantillonnage des sondes énergétiques du cluster ou à des fluctuations ponctuelles de la charge système, rendant l'interprétation fine de cette métrique délicate sur des temps d'exécution courts.

5.4 Bilan

Cette configuration offre un compromis intermédiaire. Elle évite l'effondrement des performances observé sur certains cas GPU mal dimensionnés. Elle est adaptée pour des calculs de taille moyenne où la simplicité de mise en œuvre prime sur l'optimisation absolue.

6 Comparaison des Architectures : CPU vs GPU

Pour conclure cette étude, nous comparons les meilleures performances obtenues sur chaque architecture. Nous sélectionnons pour chaque taille de maillage la configuration (nombre de nœuds ou de GPU) ayant donné le **meilleur temps de restitution**.

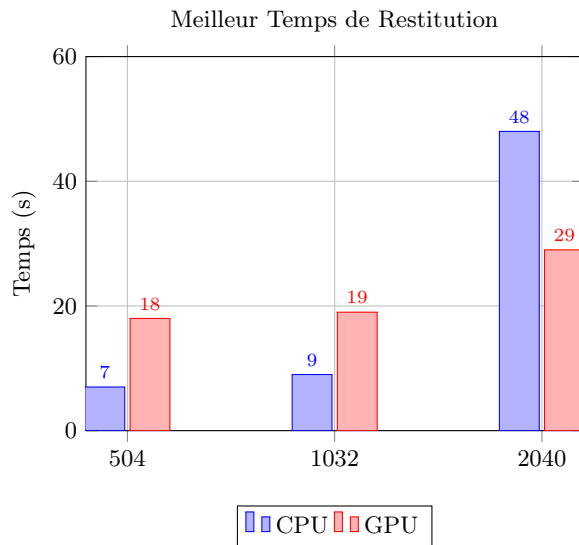


FIGURE 11 – Comparaison des Temps.

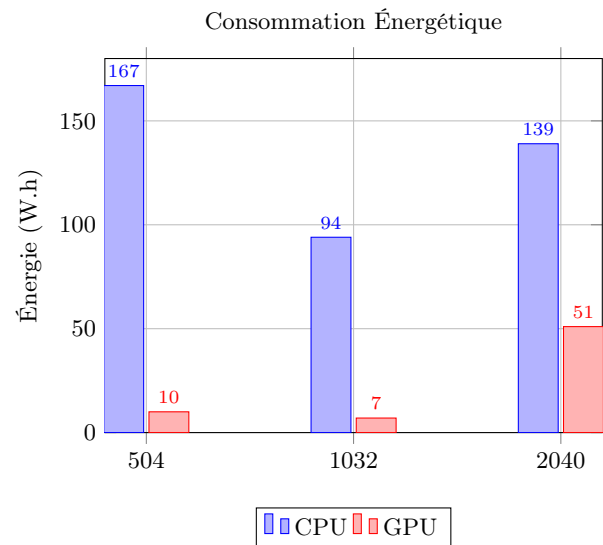


FIGURE 12 – Comparaison Énergie (Plus bas est mieux).

6.1 Bilan comparatif

L'analyse croisée des meilleures configurations sur chaque architecture permet de dégager des tendances claires :

1. **Vitesse de calcul (Temps) :** Une inversion de performance s'opère en fonction de la taille du problème.
 - Sur les maillages petits et moyens ($Nx = 504, 1032$), le cluster CPU reste le plus rapide (ex : 9s contre 19s pour $Nx = 1032$), profitant d'une parallélisation massive efficace sur des données qui tiennent en cache.
 - Sur le grand maillage ($Nx = 2040$), la tendance s'inverse : le GPU prend l'avantage (29s) face au CPU qui peine davantage (48s). L'architecture GPU montre ici sa supériorité pour traiter les très grandes matrices.
2. **Efficacité Énergétique :** Le GPU est drastiquement plus efficace, quelle que soit la taille du problème.
 - Sur les petits cas, le CPU consomme une grande quantité d'énergie pour gagner quelques secondes, là où le GPU ne consomme que 10 W.h.
 - Sur le grand cas ($Nx = 2040$), même si la consommation du GPU augmente (51 W.h), elle reste près de 3 fois inférieure à celle du CPU (139 W.h).

Conclusion : Le choix de l'architecture dépend de la priorité fixée. Si l'objectif est la vitesse absolue sur de petits problèmes, le CPU est préférable. Cependant, pour des problèmes de grande dimension ($Nx \geq 2040$) ou si l'efficacité énergétique est un critère, **le GPU est la solution optimale**, offrant le meilleur temps de calcul tout en divisant la facture énergétique par un facteur allant de 3 à 15.